ETH
Eidgenössische Technische Hochschule Zürich
Swiss Federal Institute of Technology Zurich

Institut für Integrierte Systeme
Integrated Systems Laboratory

Department of Information Technology and
Electrical Engineering

Spring Semester 2026

Exploration of On-Chip Link Width in the Gwaihir SoC

Semester Project

Lars Kröger
lkroeger@student.ethz.ch

June 2026

Supervisors: Lorenzo Leone, lleone@iis.ee.ethz.ch
Luca Colagrande, colluca@iis.ee.ethz.ch
Professor: Prof. Dr. Luca Benini, lbenini@iis.ee.ethz.ch

Abstract

This work evaluates how increasing the wide Network-on-Chip (NoC) data path affects the Gwaihir System-on-a-Chip (SoC), a 4×4 Snitch-cluster-based many-core accelerator using FlooNoC. Starting from the 512-bit baseline, multiple Direct Memory Access (DMA)/NoC width configurations up to 4096-bit NoC links are explored. The goal is to understand how much performance can be gained for memory-bound kernels, and where the architectural and physical-design limits of wider links appear.

The evaluated workloads are read-only traffic, Alpha X plus Y (AXPY), and General Matrix-Vector Multiplication (GEMV), which are all limited by memory bandwidth in the tested configurations. A roofline-based model is used to explain the expected bandwidth bottleneck for different compute-cluster-to-memory-tile ratios, and the benchmarks are then run with 4, 8, and 16 active clusters sharing one memory tile. To avoid scaling the full Snitch cluster datapath with the NoC, this work introduces a Wide Link Adapter that decouples the cluster-side DMA width from the external NoC width.

The physical-design results show that this decoupling is important. Increasing the NoC width to 1024 bit is still feasible with a moderate area increase, while larger widths quickly increase routing pressure and prevent timing closure. In particular, configurations that also increase the internal DMA width are more expensive and harder to route than configurations where only the external NoC width is increased.

The performance results show that wider NoC links can bring substantial speedups, but also that raw width scaling alone is not enough. A narrow cluster-side DMA interface can create backpressure at the memory tile, especially for long read bursts. To address this, a read-side memory-tile interleaver is implemented, which improves performance on average by 49.5% for the read-only kernel, 9.2% for AXPY, and 15.8% for GEMV. Overall, this work shows that scaling FlooNoC link width is feasible and useful for memory-bound kernels, but only if the NoC width, cluster-side DMA width, and memory-tile scheduling are considered together.

Eidgenössische Technische Hochschule Zürich
Swiss Federal Institute of Technology Zurich

Declaration of originality

The signed declaration of originality is a component of every written paper or thesis authored during the course of studies. **In consultation with the supervisor**, one of the following two options must be selected:

☐ I hereby declare that I authored the work in question independently, i.e. that no one helped me to author it. Suggestions from the supervisor regarding language and content are excepted. I used no generative artificial intelligence technologies[1].

☒ I hereby declare that I authored the work in question independently. In doing so I only used the authorised aids, which included suggestions from the supervisor regarding language and content and generative artificial intelligence technologies. The use of the latter and the respective source declarations proceeded in consultation with the supervisor.

Title of paper or thesis:

Exploration of On-Chip Link Width in the GuaihirSoC

Authored by:
If the work was compiled in a group, the names of all authors are required.

Last name(s):	**First name(s):**
Kröger	Lars Andreas

With my signature I confirm the following:
- I have adhered to the rules set out in the Citation Guidelines.
- I have documented all methods, data and processes truthfully and fully.
- I have mentioned all persons who were significant facilitators of the work.

I am aware that the work may be screened electronically for originality.

Place, date	**Signature(s)**
~~17.06.~~	
Zürich, 17.06.2026	

If the work was compiled in a group, the names of all authors are required. Through their signatures they vouch jointly for the entire content of the written work.

[1] For further information please consult the ETH Zurich websites, e.g. https://ethz.ch/en/the-eth-zurich/education/ai-in-education.html and https://library.ethz.ch/en/researching-and-publishing/scientific-writing-at-eth-zurich.html (subject to change).

Contents

List of Acronyms **viii**

1. Introduction **1**

2. Related Work **3**
2.1. Many-Core Accelerator Architectures . . . 3
2.2. Snitch-Based Many-Core Systems . . . 3
2.3. FlooNoC . . . 4

3. Theory / Algorithms **6**
3.1. Model . . . 6
3.2. Algorithmic implementation . . . 8

4. Hardware Architecture **9**
4.1. The AXI4 Protocol . . . 9
4.2. Wide Link Adapter in the Cluster Tile . . . 10
4.3. Interleaving at the Memory Tile . . . 10
4.3.1. Memory Controller Architecture . . . 12
4.3.2. Read-Side Interleaving . . . 12
4.3.3. Write-Side Interleaving . . . 13

5. Design Implementation and Results **16**
5.1. Floorplan and Pins . . . 16
5.2. Backend Flow . . . 17
5.3. Snitch Cluster Tile . . . 18
5.3.1. Area . . . 18
5.3.2. Timing and Congestion . . . 20
5.4. Memory Tile . . . 23

6. Performance Results **26**
6.1. Impact of Memory-Tile Interleaving . . . 28

7. Conclusion and Future Work **31**

A. Task Description **33**

B. Full Area Breakdown **35**

List of Figures

2.1. Simplified block diagram of the Snitch cluster tile 4

3.1. Roofline plot for compute-tile to memory-tile ratios from 2 clusters/tile to 32 clusters/tile for AXPY, GEMV, and autoregressive inference. 7
3.2. Memory bottleneck in the GEMV and AXPY kernels in Gwaihir. 8

4.1. Advanced eXtensible Interface (AXI) Wide Link Adapter. 11
4.2. Memory controller in Gwaihir. 12
4.3. Architecture of Read Side Controller in the Memory Tile 14

5.1. Area results for the evaluated Snitch cluster-tile configurations. The left plot shows the total cluster-tile area split into base cluster logic and width-scaled modules. The right plot shows the area breakdown of the modules that scale with increased data width. 19
5.2. Hierarchical die shots of the evaluated Snitch cluster-tile configurations. . 21
5.3. Routing congestion maps for the evaluated Snitch cluster-tile configurations. 22
5.4. Area results for the evaluated memory-tile configurations. 24

6.1. Performance speedups for the evaluated compute-cluster-to-memory-tile ratios. Speedups are normalized to the 512-bit DMA / 512-bit NoC baseline. 27
6.2. Performance speedups for non-interleaving (NI) and interleaving configurations for the evaluated compute-cluster-to-memory-tile ratios. 29

List of Acronyms

AI . . . . . Arithmetic Intensity

AR . . . . . Read Address

ASIC . . . . . Application-Specific Integrated Circuit

AW . . . . . Write Address

AXI . . . . . Advanced eXtensible Interface

AXPY . . . . . Alpha X plus Y

CTS . . . . . Clock Tree Synthesis

DMA . . . . . Direct Memory Access

DRC . . . . . Design Rule Check

DVI . . . . . Device Independent File Format

ECO . . . . . Engineering Change Order

EPS . . . . . Encapsulated PostScript

FLOP . . . . . Floating-Point Operations

FMADD . . . . Floating-point fused multiply-add

fMAX . . . . . Maximum Frequency

FPU . . . . . Floating-Point Unit

GEMV . . . . . General Matrix-Vector Multiplication

IC . . . . . Integrated Circuit

kGE . . . . . kilo Gate Equivalent

NI . . . . . Network Interface

NoC . . . . . Network-on-Chip

OBI . . . . . Open Bus Interface

PnR . . . . . Place and Route

RTL . . . . . Register Transfer Level

SoC . . . . . System-on-a-Chip

SRAM . . . . . Static random-access memory

TCDM . . . . . Tightly Coupled Data Memory

TNS . . . . . Total Negative Slack

WNS . . . . . Worst Negative Slack

Chapter 1

Introduction

Many-core accelerator architectures expose a large amount of spatial parallelism by distributing computation over many compute tiles. This is especially useful for machine-learning workloads, where a large number of operations can often be executed in parallel across different parts of the data. However, the available compute throughput only translates into useful performance if the memory system and the on-chip interconnect can provide the required data rate. As the number of compute tiles increases, the aggregate memory traffic also increases, and the NoC becomes one of the main components that determines how well the system can actually scale.

This is particularly important for kernels with low arithmetic intensity. AXPY and GEMV, for example, perform only a small number of floating-point operations for every byte moved from memory. Their performance is therefore limited by the available memory bandwidth much earlier than by the peak compute throughput of the cores. Similar behavior also appears in autoregressive inference, where token-by-token execution leads to repeated low-reuse linear algebra and continuous pressure on the memory system. For these workloads, increasing compute alone is not sufficient. The bandwidth between memory tiles and compute tiles has to scale as well.

This work studies this problem in the Gwaihir SoC, a 4×4 many-core accelerator based on Snitch clusters and the open-source FlooNoC [1] interconnect. The baseline system uses a 512-bit wide NoC data path. The main goal of this project is to explore how increasing this link width affects performance for memory-bound kernels, and which architectural and physical-design costs are introduced by doing so. The physical-design evaluation is carried out in a commercial TSMC 7nm technology, making the area, timing, congestion, and pin-pressure results representative of a modern implementation flow.

Simply increasing the data width everywhere in the compute tile is not necessarily a good solution. Wider datapaths increase area, routing pressure, tile-boundary pin count, and timing difficulty inside the Snitch cluster. Therefore, this work separates the internal

DMA width of the cluster from the external NoC width. A Wide Link Adapter is inserted between the Snitch cluster and the Network Interface (NI), allowing the cluster to keep a narrower local data path while the NoC operates at a larger width. Since this creates a mismatch between the rate at which the memory tile can return data and the rate at which a cluster can drain it, this work also implements read-side interleaving in the memory tile.

The main contributions of this project are:

- a roofline-based bandwidth model for AXPY, GEMV, and autoregressive-inference-like traffic under different compute-cluster-to-memory-tile ratios;
- a parameterizable Wide Link Adapter that decouples the cluster-side DMA width from the NoC link width;
- a read-side memory-tile interleaver that allows multiple clusters to be served in parallel at the burst level;
- a backend evaluation in TSMC 7nm of area, timing, congestion, and pin-bundle size for different DMA/NoC width configurations;
- a performance evaluation showing how wider NoC links and memory-tile interleaving affect memory-bound kernels.

The remainder of this report is structured as follows. Chapter 2 introduces the related many-core systems, Snitch-based architectures, and FlooNoC. Chapter 3 introduces the bandwidth model and the evaluated kernels. Chapter 4 describes the Wide Link Adapter and the memory-tile interleaving architecture. Chapter 5 evaluates the physical-design impact of the different link-width configurations. Chapter 6 presents the benchmark results. Finally, Chapter 7 summarizes the main findings and discusses possible future work.

Chapter 2

Related Work

2.1. Many-Core Accelerator Architectures

Modern machine-learning accelerators increasingly use many-core architectures, where computation is distributed over many relatively small processing elements or compute tiles. This allows the architecture to exploit large amounts of parallelism, but it also shifts a significant part of the design challenge to data movement. The compute tiles have to be supplied with operands, partial results have to be exchanged, and memory traffic from many tiles has to be handled at the same time. The on-chip interconnect is therefore not only used for control traffic, but becomes part of the main performance-critical data path.

This trend can be seen in commercial accelerators such as the Graphcore IPU [2] and Cerebras WSE[3], but also in academic many-core systems such as MemPool[4] and Manticore[5]. While these designs target different scales and workloads, they all illustrate the same architectural pressure: increasing the number of on-chip processing elements also increases the amount of parallel memory and communication traffic that has to be handled by the on-chip interconnect.

2.2. Snitch-Based Many-Core Systems

Snitch is an ultra-small RISC-V compute core with an attached Floating-Point Unit (FPU) [6]. It aims to accelerate especially machine-learning and data-analysis algorithms by using Stream Semantic Registers to feed new data to the FPU, freeing up the compute core for other tasks. It focuses on high energy efficiency and an extremely small area footprint [6].

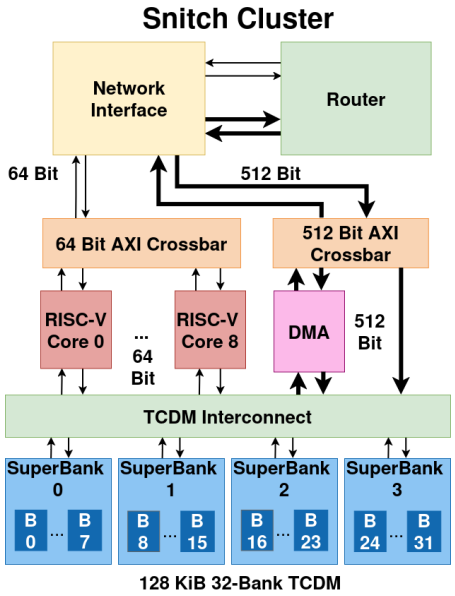


Figure 2.1.: Simplified block diagram of the Snitch cluster tile

Eight of these Snitch cores are then placed in a Snitch cluster, which features a DMA for wide data movement, a Tightly Coupled Data Memory (TCDM), as well as a NI and router for the NoC. A simplified block diagram of this Snitch cluster, showing the parts that are relevant for this work, can be seen in Figure 2.1.

Gwaihir is a many-core accelerator that is in development at the time of this report. It features a 4×4 mesh of Snitch-cluster compute tiles as well as four L2 memory tiles. This work performs all benchmarking on this SoC.

2.3. FlooNoC

FlooNoC [1] is an open-source NoC designed for bulk data transfers in many-core machine-learning accelerators. It utilizes a multi-stream DMA to route wide AXI4 packets between manager and subordinate ports in the SoC. It is focused on low area overhead and energy efficiency [1]. Architecturally, the NoC transfers packets through a router, which is present in every tile in the SoC mesh and has input and output ports for the north, south, east, and west directions. It features separate ports for wide bulk-data traffic and latency-sensitive narrow AXI traffic coming from the Snitch cluster. It is important to note here that the routing inside FlooNoC is mostly AXI-agnostic. The AXI data is simply transferred as payload in so-called flits and is not directly used in the routing protocol. The incoming and outgoing AXI requests and responses are converted into these flits in the NI, also called Chimney. This protocol conversion is implemented to adhere to the strict ordering requirements imposed by AXI4. This is done either by a reorder

buffer that keeps track of outstanding transactions with the same AXI ID to different destinations, injecting them into the network only if there is sufficient space available to store the response, or by simply stalling these overlapping transactions. The flits then travel through the compute tiles of the mesh to their final destination NI.

FlooNoC [1] was designed with configurable wide and narrow data widths, set to 512 bit and 64 bit, respectively, in the original work. This configurability is the main focus of this project. We increase the wide data width, perform detailed performance benchmarking, propose architectural changes to support the larger data paths, and run full backend flows for the evaluated configurations.

Chapter 3

Theory / Algorithms

3.1. Model

To understand how the NoC link width scales the overall performance of the Gwaihir SoC, and of many-core accelerator architectures in general, a thorough analysis of the Arithmetic Intensity (AI) and peak compute is needed. This work focuses on a few benchmarks known to be memory-bound, namely AXPY and GEMV. The compute tile meshes evaluated in this project are based on Snitch clusters, featuring eight compute cores each capable of one 64-bit Floating-point fused multiply-add (FMADD) operation per cycle. Since the FMADD operation corresponds to two Floating-Point Operations (FLOP), each compute tile will therefore have a peak compute of

$$P_{\text{tile}} = 8 \text{ cores} \cdot 2 \frac{FLOP}{\text{cycle}} = 16 \frac{FLOP}{\text{cycle}}.$$

The bandwidth demand of the SoC is then determined by the ratio between compute tiles and memory tiles, as each memory tile can supply data at the rate of the NoC data width independently. In the Gwaihir SoC, this ratio is four compute tiles per memory tile. However, as described in 3, many-core SoCs are scaling well beyond this ratio, so this work benchmarks kernels at up to a 16-to-1 ratio, which simply corresponds to having one active memory tile in the 16-cluster Gwaihir SoC.

The AXPY kernel computes

$$z = a \cdot x + y.$$

Every operation therefore takes one FMADD, so two FLOP, and needs to load two 64-bit operands from memory and write back one 64-bit operand. This corresponds to an AI of

$$I_{AXPY} = \frac{2\ FLOP}{3 \cdot 8\ \text{B}} = 0.083 \frac{FLOP}{\text{B}}.$$

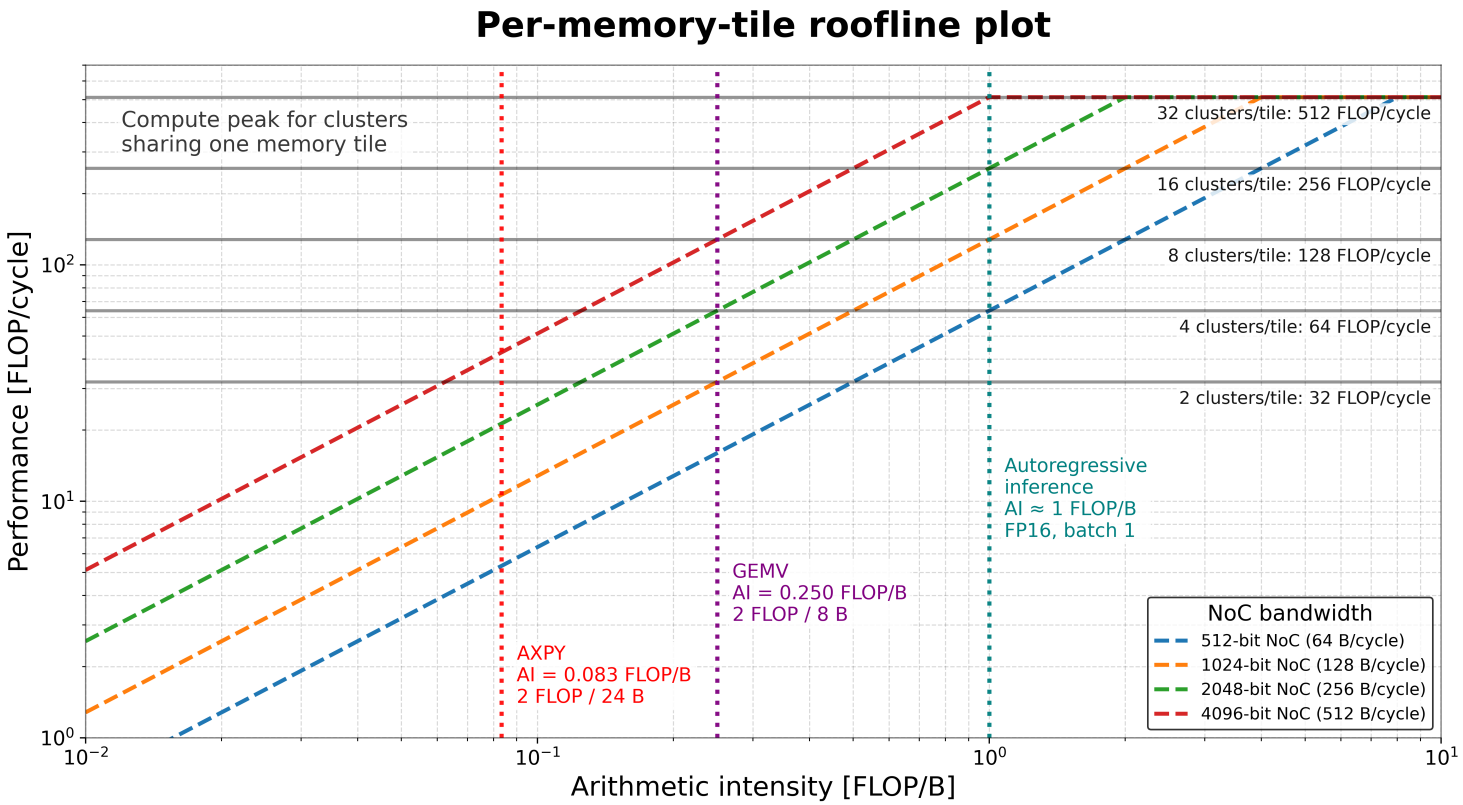


Figure 3.1.: Roofline plot for compute-tile to memory-tile ratios from 2 clusters/tile to 32 clusters/tile for AXPY, GEMV, and autoregressive inference.

GEMV, on the other hand, is

$$y = \alpha \cdot Ax,$$

where A is an $m \times n$ matrix and x and y are n-element vectors. In a Snitch cluster, each compute core then computes $m/8$ rows, each corresponding to n FMADD operations, of this matrix-vector multiplication. While a single operation is equivalent to AXPY and is one FMADD, so two FLOP, for large matrices, such as the 80×80 matrix used to benchmark in this work, the data demand per operation goes asymptotically to just 8 bytes, because the same x value is loaded and the y value is written back to memory for all n elements in a row. In the end, this gives an arithmetic intensity of

$$I_{GEMV} = \frac{2\ FLOPs}{8\ \text{B}} = 0.25\ \frac{FLOP}{\text{B}}.$$

Figure 3.1 shows the roofline plot for cluster-tile per memory-tile ratios from 2 clus-

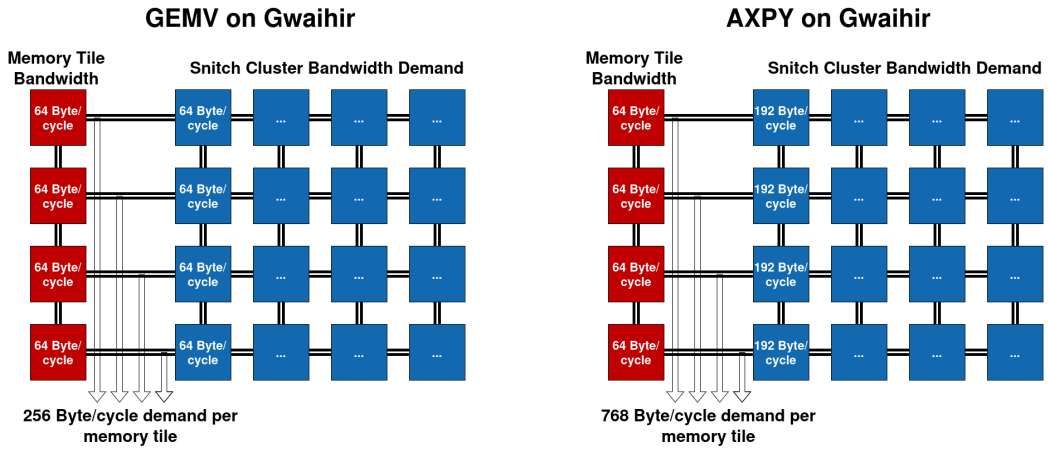


Figure 3.2.: Memory bottleneck in the GEMV and AXPY kernels in Gwaihir.

ters/tile to 32 clusters/tile for AXPY, GEMV, and autoregressive inference, which is also said to have low arithmetic intensity [7]. The NoC configurations range from 512-bit to 4096-bit data width. The plot shows how deeply memory-bound these kernels are in many-core architectures. AXPY does not become compute-bound even for a 4096-bit link width, and GEMV only becomes compute-bound at a 2048-bit link width. The bandwidth demand mismatch in the Gwaihir mesh at the 512-bit link width it was operating at before this project can also be seen in Figure 3.2

3.2. Algorithmic implementation

In order to see speedups in line with the scaling factor of the NoC, the available NoC bandwidth must be fully utilized. For this reason, all workloads use double buffering for input and output data. While the computation is performed on the current tile, the next tile is loaded in parallel.

Another important aspect when benchmarking is AXI burst alignment. The AXI protocol does not support read or write bursts across 4 KiB address boundaries. This means that if data is not aligned to these boundaries, multiple individual read or write requests may be required for what would otherwise be one 4 KiB burst. Typically, this results in one large burst and one smaller burst for the remaining beats up to the next 4 KiB boundary. This increases protocol overhead, especially at larger link widths where each 4 KiB burst contains fewer data beats. For this reason, all data buffers located in the local TCDM and in L2 memory are aligned to 4 KiB boundaries.

Chapter 4

Hardware Architecture

This work aims to increase the effective data rate that can be read/written to/from the memory tile to improve performance in memory-bound kernels. However, simply increasing the wide data width used throughout the architecture is not an isolated design change. It affects the interfaces around the NoC, the AXI protocol assumptions, the DMA interface inside the cluster, and the way memory traffic is served at the memory tile. Therefore, several architectural decisions have to be made before the NoC can be scaled to wider data paths.

4.1. The AXI4 Protocol

One of the first limitations comes from the AXI4 protocol itself. AXI4 was not designed for arbitrarily large data widths. The transfer size is encoded in the `size` field, which is only 3 bits wide [8]. This means that the largest AXI-compliant transfer size is 2^7 bytes, corresponding to a maximum data width of 1024 bits.

It is simple to break this limitation by manually extending the `size` field to 4 bits. This would allow transfer sizes up to 2^{15} bytes and therefore much larger link widths. The NoC functionality is mostly unaffected by this since it is mostly AXI-agnostic. It routes packets as flits and simply carries the AXI transaction information as payload. However, extending the AXI fields would make the interface non-standard and could cause compatibility issues with existing or future AXI peripherals. The DMA in the Snitch cluster, for example, does not work with this extended AXI package out-of-the box. This, however, is most likely a simple size mismatch/truncation issue in a module that hard-codes the AXI size field definition directly, or as an assumption somewhere, and therefore fixable by adjusted package definitions.

4.2. Wide Link Adapter in the Cluster Tile

More importantly, however, as we will see during the physical design evaluation in Chapter 5, increasing the link width in the Snitch cluster itself degrades timing and increases routing congestion. A backend run where both the Snitch cluster as well as the NoC were configured to have a 2048-bit data width, for example, could not complete routing with the routing resources available. The theoretical model of AXPY and GEMV in Section 3.1 also shows that because we have more cluster tiles than memory tiles, we in theory do not need to have the clusters themselves running at the full data width of the NoC.

If we make a few important architectural decisions around the memory controller, as we do in Section 4.3, we can utilize the full bandwidth of the NoC as long as the aggregate bandwidth of all clusters per memory tile exceeds that of the NoC. For the Gwaihir SoC, this would correspond to a data width in the NoC that is at most four times larger than the data width inside the DMA and Snitch cluster.

To decouple these two data widths, this work introduces a Wide Link Adapter, placed between the NI and the Snitch cluster in the compute tile. The architecture can be seen in Figure 4.1 for an example configuration of 512-bit link width in the Snitch cluster and 1024-bit width in the NoC, however it is parameterizable to work with any combination of these widths. It is based on existing AXI data width converters [9]. The adapter is placed between the NI and the Snitch cluster and features converters on the manager and subordinate AXI ports.

For the manager port, the returned read data is buffered and then split into beats of the Snitch cluster link width and drained beat by beat. Write data is also buffered and merged into a wide data beat. The length, burst size, and address alignment of the Read Address (AR) and Write Address (AW) are also recalculated to fit the wider link width. The narrow subordinate side, on the other hand, accumulates the read data on the AXI RSP path and splits large write beats in the REQ path.

The adapter works on a per-AXI-ID basis and can be configured to feature a variable number of these up- and downsizers to handle multiple AXI requests with unique IDs in parallel. This of course raises the cost in terms of sequential and combinational logic the adapter instantiates, which can become quite significant, as seen in Chapter 5, since the full data width of the wide path is instantiated twice per manager/subordinate AXI port and converter. The adapter is also configurable to use the increased size field that extends the AXI protocol past 1024-bit data width on one or both sides.

4.3. Interleaving at the Memory Tile

Running the NoC at a wider data width than the DMA inside the cluster introduces another challenge. The link-width extender between the DMA and the NI can only drain

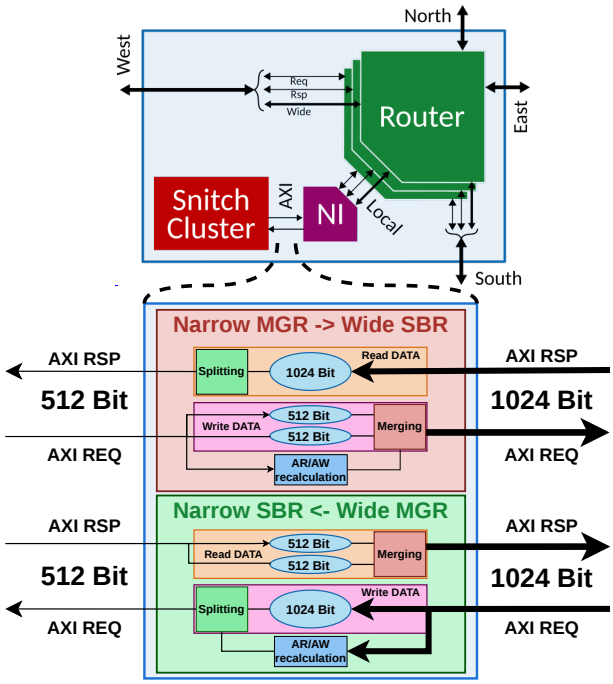


Figure 4.1.: AXI Wide Link Adapter.

one beat of the smaller DMA interface per cycle into the cluster. At the same time, it can only buffer one beat of the larger NoC data width per outstanding AXI ID request before it has to apply backpressure toward the memory tile.

This becomes especially important for long burst transfers. If the memory tile serves requests from the clusters in a purely serialized order, then a wide NoC response immediately fills the corresponding buffer in the receiving cluster. Once this happens, the memory tile is stalled until the cluster has drained the wide data beat through its smaller DMA interface. As a result, even though the NoC and the memory tile are capable of transferring data at the larger NoC width, the effective memory-tile utilization is limited to the smaller DMA data width of a single cluster.

To avoid this, the memory tile should interleave responses between different clusters or different outstanding requests. Instead of continuously serving one cluster until its full burst is complete, the memory tile should switch between multiple active requests. While one cluster is draining its previously received wide beat through the narrower DMA interface, the memory tile can already serve another cluster. This allows the wide NoC link and the memory tile interface to be utilized fully, even though each individual cluster still consumes data at the smaller DMA width.

4.3.1. Memory Controller Architecture

The memory controller architecture in Gwaihir can be seen in Figure 4.2. Incoming wide AXI requests coming from the router are remapped to ID 0 in the meta buffer of the NI. The wide and narrow AXI ports are then merged into a single AXI port operating at the wide data width at which the memory tile is also operating. Here, the IDs of the requests are again reconfigured to preserve the lower IDs for atomic transactions on the narrow port. There are then two independent memory controllers for read and write transactions, which each accept one AW/AR at a time and apply backpressure until the transaction has been completed. The AXI request is then translated to the Open Bus Interface (OBI) protocol used by the Static random-access memory (SRAM). The OBI streams from the read and write side are multiplexed each cycle.

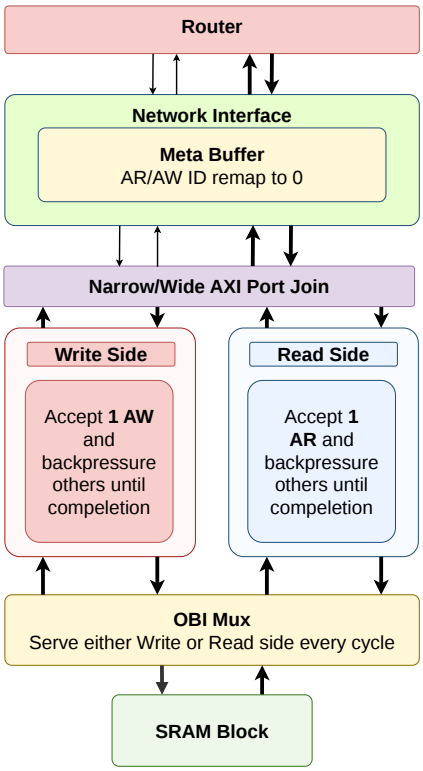


Figure 4.2.: Memory controller in Gwaihir.

4.3.2. Read-Side Interleaving

Since memory traffic from the kernels described in Section 3.1 is read-heavy, serializing the long ARs from different Snitch clusters significantly degrades performance, as shown

in Figure 6.2. To address this, we implement a round-robin interleaving architecture for the read side of the memory controller.

The architecture of the updated read-side block can be seen in Figure 4.3, in the context of Figure 4.2. The new controller can now store a configurable number of ARs, and a round-robin arbiter with a fair arbitration policy picks an AR for which a read data beat should be returned in each cycle. The controller then translates the AXI request to OBI, which is then multiplexed to the SRAM as described before in Section 4.3. This way, each Snitch cluster operating at a lower data width has enough time to drain the multiple narrow data beats for each wide data beat provided by the memory tile, while the memory tile serves a wide data beat to another cluster.

The AR slots need to store the full metadata of each AR, including the size, length, and burst increment. A "beats remaining" counter then tracks the progress of each AR. The read response, containing the data beat as well as the ID and an RLAST field signaling completion of the full transaction, is also formatted using this metadata. It is important to note that the read data itself is not buffered in this architecture, which is important to reduce the amount of sequential logic introduced by this interleaving configuration.

To utilize the different slots, the ARs need to carry different IDs when they arrive at the slot allocator. In the configuration from Section 4.3, where each AXI ID was remapped to 0 in the NI, the updated architecture would therefore be unable to distinguish requests from different clusters and could not interleave them effectively. Since the DMAs in the Snitch clusters only use two unique IDs for their ARs, which are identical between different clusters, using these IDs, which are available in the metadata of the flits arriving from the router, is not an option.

The metadata of each flit also carries the x and y coordinate of the source of each AR in the mesh of the SoC, so the 4×4 compute tile mesh with four memory tiles in Gwaihir. We remap the IDs of the ARs in the meta buffer of the NI to the flattened version of this source ID. To ensure no clipping, we also extend the ID field, which has configurable size in AXI, to 5 bits.

Through this configuration, we also adhere to the strict ordering requirements imposed by the AXI protocol. Since the AR slot allocator will only accept one outstanding AR per unique ID, there is strict ordering between ARs originating from the same cluster. The meta buffer also stores the information required to reverse the ID remapping for each AR in a keyed buffer, so the flits returning to the cluster carry back the original ID.

4.3.3. Write-Side Interleaving

For write-heavy kernels such as AXPY, it would also be useful to interleave write data. Ideally, the write-side memory controller would accept multiple AW requests and then map each incoming W beat to its corresponding AW. This is more difficult than on the

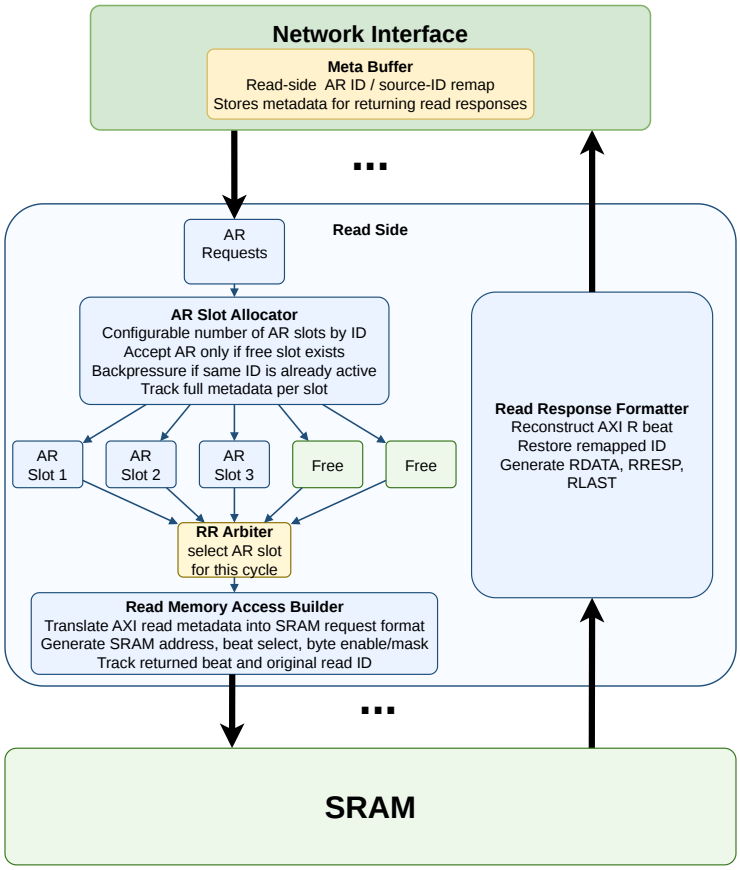


Figure 4.3.: Architecture of Read Side Controller in the Memory Tile

read side, because write data beats in AXI4 no longer carry an ID field. This ID field, and with it the official support for write-data interleaving, existed in AXI3 but was removed when moving to AXI4.

As part of this work, we implemented write-side interleaving in the memory controller using custom user bits to map each W beat back to its corresponding AW. However, this could not be used effectively in the current system, since the router currently enforces strict serialization of AW requests. Once one AW is in flight, the router locks this route and does not route another AW until the transaction has completed.

Using the write-side interleaver would therefore require changes to the router architecture. One possible solution would be to allow interleaved AW requests only for transactions targeting the memory tile, while keeping the stricter behavior for other destinations where the ordering requirements are more difficult to guarantee.

Chapter 5

Design Implementation and Results

The previous chapters motivate wider NoC links from a performance perspective and introduced the architectural changes required to use this bandwidth effectively. However, increasing the available data width is not only a system-level performance decision. It also changes the physical implementation of the design, since wider links require more routing resources and increase area usage.

This chapter therefore evaluates the proposed configurations through the backend flow in the TSMC 7nm technology, using the fusion compiler. The goal is to understand how scaling the NoC link width affects area, timing and routing congestion. In addition, the evaluation separates the impact of increasing the data width inside the Snitch cluster from the impact of increasing the NoC data width itself. This makes it possible to identify which parts of the design scale well, which parts become physically limiting, and where the Wide Link Adapter and memory-tile interleaving logic introduce additional implementation cost.

5.1. Floorplan and Pins

The router in the cluster tile forwards packets in the east, west, north, and south directions. In each direction, the router has both incoming and outgoing links, and each link carries AXI request and response channels. The dominant contribution to the number of pins at each tile boundary comes from the wide data payload itself.

For a link width of W_{data}, each directional router port has one incoming and one outgoing link. Each of these links carries one wide data field on the request path and one wide

data field on the response path. Therefore, the wide data signals alone require

$$N_{\text{pins,data}} = \underbrace{2}_{\text{REQ/RSP data paths}} \cdot \underbrace{2}_{\text{input/output links}} \cdot W_{\text{data}} = 4 \cdot W_{\text{data}}$$

pins per router direction.

Although Gwaihir has not been taped out at the time of this report, the Snitch cluster tiles were taped out in Picobello [10] with dimensions of 697.68 μm $\times$ 921.12 μm. For this work, the exact number of tile-boundary pins is extracted from the implemented cluster-tile configurations after changing the NoC link width. These pin counts therefore include not only the wide data payload, but also the additional request and response signals, routing metadata, address, strobe, control, and handshake signals.

As was the case in Picobello, we have four metal layers available for routing these pins, namely M5 to M8. M5 and M7 are used in the vertical direction, while M6 and M8 are used in the horizontal direction. Each layer has a minimum routing pitch of 0.076 μm. Since two metal layers are available per routing direction, the required pin-bundle length is calculated as

$$L_{\text{pins}} = \frac{N_{\text{pins}} \cdot 0.076\ \mu\text{m}}{2}.$$

The resulting values are shown in Table 5.1. The length represents a lower bound, since it assumes minimum-pitch placement without spacing the pins multiple tracks apart.

Table 5.1.: Per-direction pin count and minimum pin-track length for different NoC link-width configurations.

NoC data width	**Pins**	**Minimum pin-bundle length**
512 bit	2948	112.0μm
1024 bit	5252	199.6μm
2048 bit	9860	374.7μm
4096 bit	19076	724.9μm

This shows that the 4096-bit configuration no longer fits within the Picobello cluster tile dimensions. It also shows that moving to even larger widths, such as 8192 bit, would not be feasible for the compute tiles without substantially increasing the tile dimensions or using more routing layers for the pins.

5.2. Backend Flow

We used the Fusion Compiler to run the full backend flow for different configurations. Since this work is aimed at design-space exploration, we omitted steps such as Engineering Change Order (ECO) optimization, manual fixing of Design Rule Check (DRC)/antenna

violations, and manual upsizing of cells in critical timing paths. We believe that especially some of the timing problems could be resolved in this way. For all Place and Route (PnR) runs examined below, we disabled ungrouping for modules we expect to scale with the link width, namely the NoC router, NI, DMA interconnect, and AXI crossbar.

5.3. Snitch Cluster Tile

We first want to evaluate the area impact of increasing the NoC and DMA data width both together and separately. To this end, we ran the full backend flow for the following configurations:

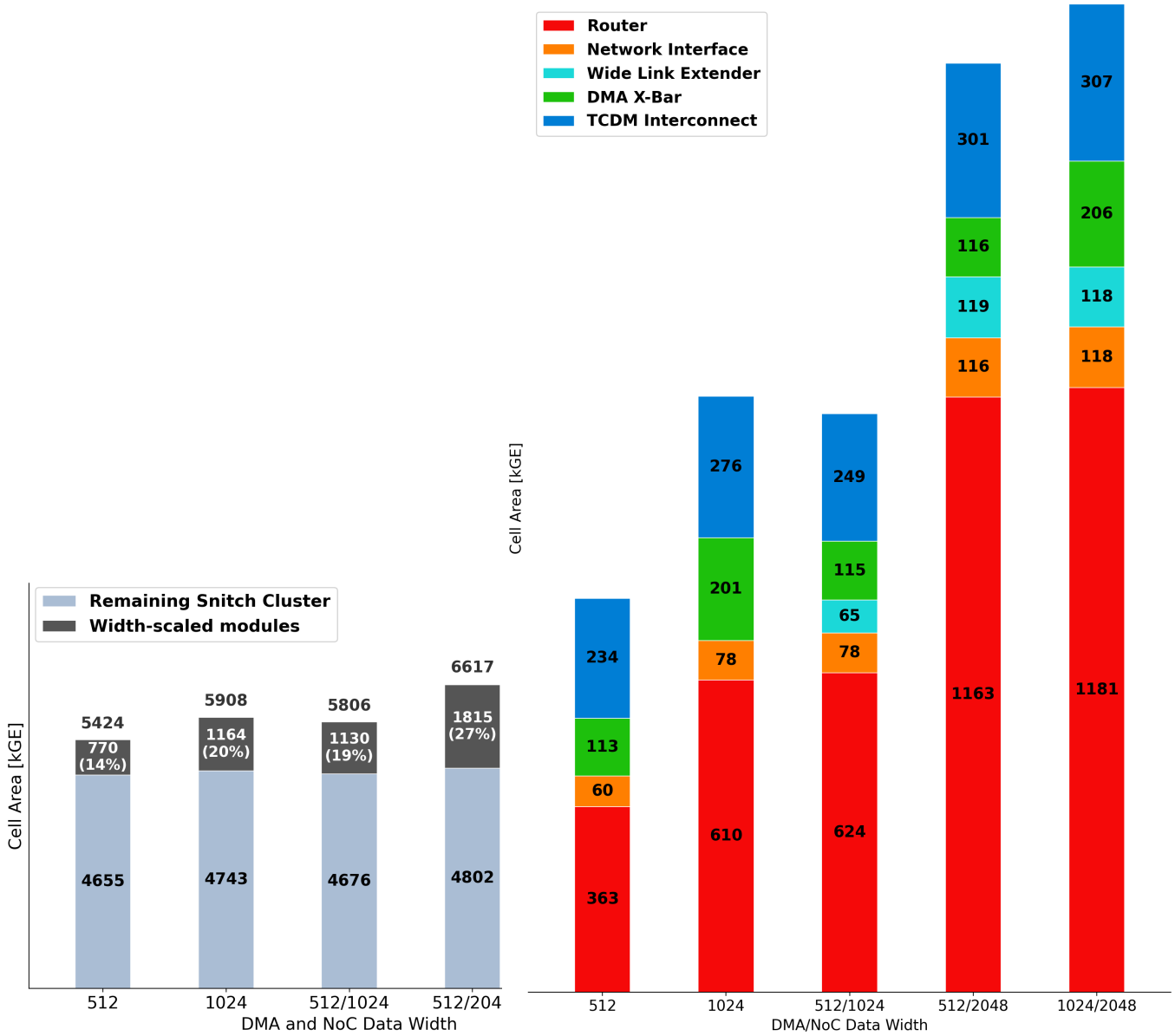
- **512-bit** NoC and **512-bit** DMA: **Baseline**,
- **1024-bit** NoC and **1024-bit** DMA,
- **1024-bit** NoC and **512-bit** DMA,
- **2048-bit** NoC and **512-bit** DMA,
- **2048-bit** NoC and **1024-bit** DMA,
- **4096-bit** NoC and **1024-bit** DMA,
- **2048-bit** NoC and **2048-bit** DMA.

Naturally, all configurations featuring different DMA and NoC widths include the Wide Link Adapter introduced in Section 4.2, in our case configured to have two parallel data width converters for the manager and subordinate AXI ports. We were able to route all designs except for the 2048-bit DMA and 2048-bit NoC configuration and the 1024-bit DMA and 4096-bit NoC configuration. The following sections therefore use the design after Clock Tree Synthesis (CTS) for these configurations.

5.3.1. Area

When increasing the data width in the Snitch cluster and NoC, we expect to increase both combinational and sequential cell area in the cluster tile. On the one hand, modules like the router and the Wide Link Adapter buffer the full data width, for the router even per direction, while the Wide Link Adapter also needs to buffer data beats to accumulate or split them. The DMA crossbar and TCDM interconnect, on the other hand, get increasingly complex with increased data width, and we expect increases in combinational cell area.

The full breakdown of the area of all configurations that we were able to fully route can be seen in Figure 5.1.



Total cluster-tile area **Width-scaled module breakdown**

Figure 5.1.: Area results for the evaluated Snitch cluster-tile configurations. The left plot shows the total cluster-tile area split into base cluster logic and width-scaled modules. The right plot shows the area breakdown of the modules that scale with increased data width.

5.2 shows these width scaled modules in a die shot for each configuration. For the all-2048-bit config and the 1024-4096 bit config these pictures are taken after CTS, so the modules will look smaller than they would be after buffer insertion in the full routing flow.

“‘latex “‘

The NoC router and the Wide Link Adapter scale almost linearly with the NoC data width, while the DMA crossbar scales with the data width inside the Snitch cluster and the DMA. The minimum total area cost of increasing the NoC width is therefore around 8% for 1024 bit and around 22% for 2048 bit. If only the modules that directly scale with the data width are considered, the increase is much larger, with 47% for the 1024-bit configuration and 235% for the 2048-bit configuration. This distinction is important for future work, since a smaller or simplified Snitch cluster would make the relative cost of the width-scaled modules much more significant.

5.3.2. Timing and Congestion

To evaluate the feasibility of higher link widths in the physical design, we evaluate their impact on timing closure and routing congestion. The congestion maps for all five routable cluster-tile configurations are shown in Figure 5.3. These maps make it possible to compare how the routing pressure changes when the NoC width, the DMA width, or both are increased.

The congestion maps show that increasing the data width increases routing pressure around the width-scaled modules and the tile boundary. This is expected, since wider NoC links add a large number of parallel wires, while a wider DMA interface also increases the width of the datapaths inside the Snitch cluster. The configurations where only the NoC width is increased are therefore less critical than configurations where the DMA width is increased as well. In particular, keeping the Snitch cluster at a lower internal data width reduces congestion inside the cluster tile and confines most of the additional routing pressure to the adapter and NoC-facing logic.

Congestion also directly affects timing closure. Table 5.2 shows the Worst Negative Slack (WNS), Total Negative Slack (TNS), and resulting Maximum Frequency (fMAX) the cluster tile can operate at in each configuration in the evaluated process corners. The table reports the results for the worst-case slow-slow corner at $-40\,^{\circ}\mathrm{C}$ and at $125\,^{\circ}\mathrm{C}$.

We can observe that especially width increases in the DMA significantly degrade timing. It should be noted, however, that for all configurations the most critical timing paths do not actually pass through the modules that are scaled by the increased data width. Instead, the critical paths are located in the program-counter path of a Snitch core itself. This means that the reported timing degradation should not be interpreted as the Wide Link Adapter or the wider NoC directly becoming the critical path. Rather, the

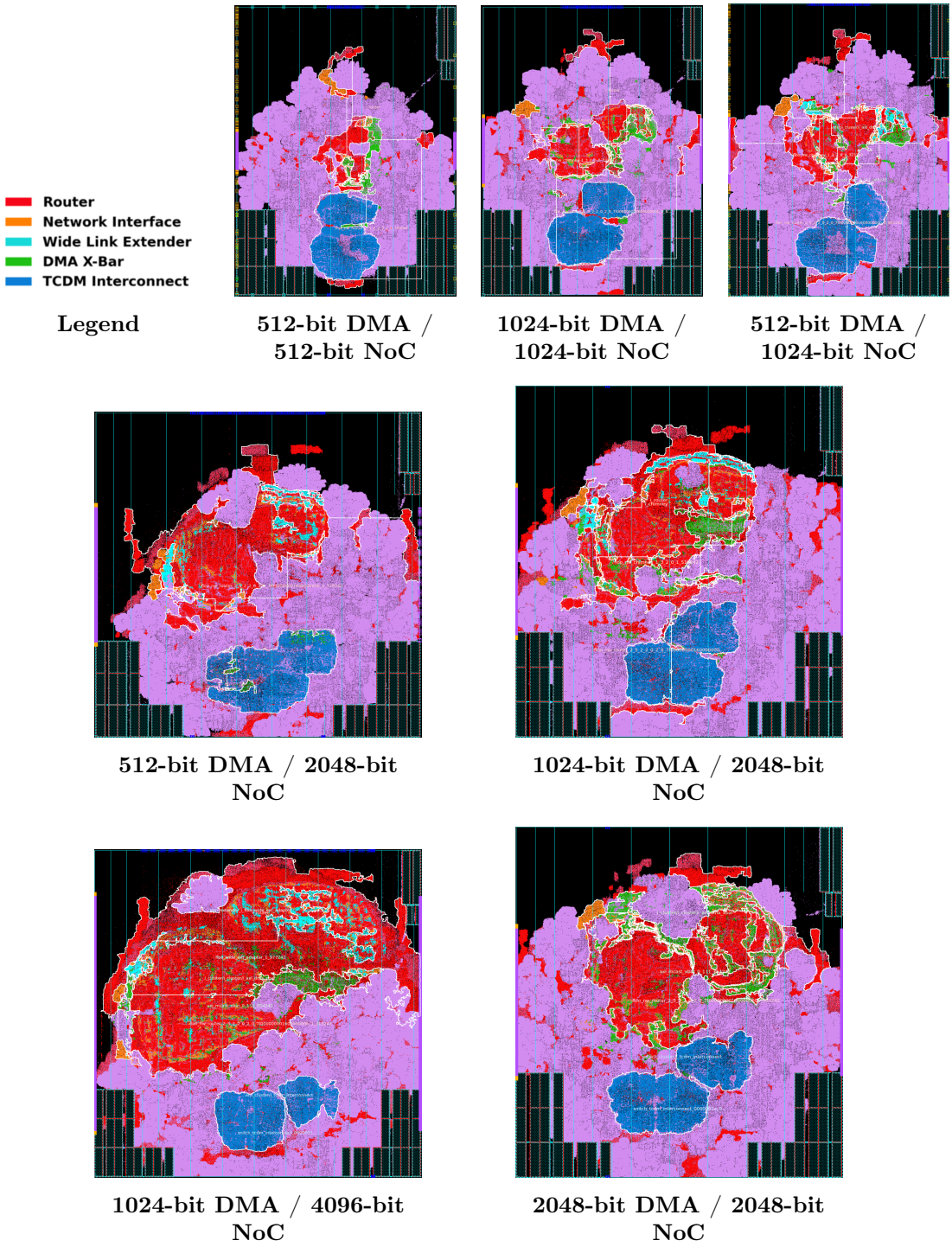
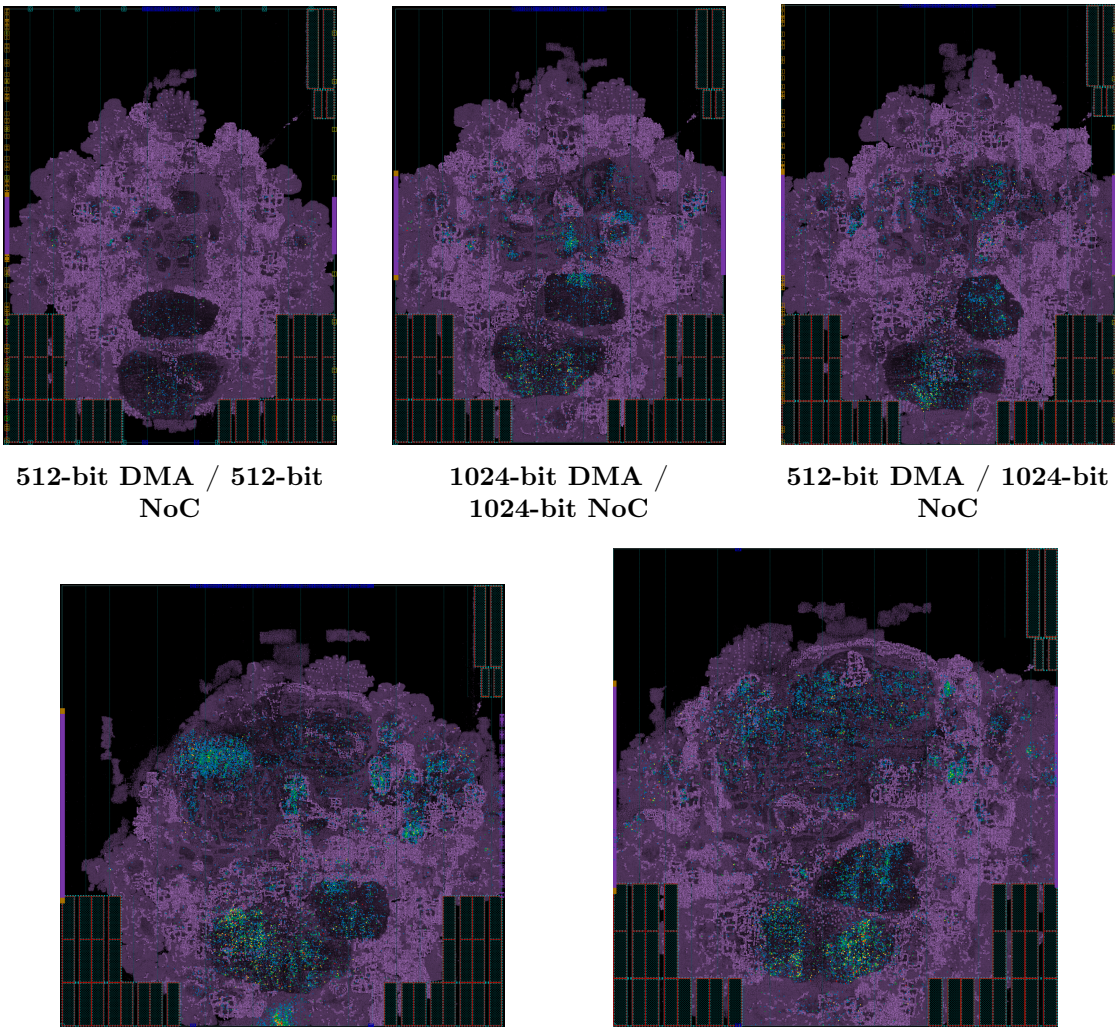


Figure 5.2.: Hierarchical die shots of the evaluated Snitch cluster-tile configurations.



512-bit DMA / 512-bit NoC

1024-bit DMA / 1024-bit NoC

512-bit DMA / 1024-bit NoC

512-bit DMA / 2048-bit NoC

1024-bit DMA / 2048-bit NoC

Figure 5.3.: Routing congestion maps for the evaluated Snitch cluster-tile configurations.

Table 5.2.: Timing results for the evaluated Snitch cluster-tile configurations in the slow-slow process corners.

Corner	DMA width	NoC width	WNS (ns)	TNS (ns)	fMAX (MHz)
SS −40 °C	**512-bit**	**512-bit**	-0.121	-9.634	892.1
		1024-bit	-0.122	-23.801	891.3
		2048-bit	-0.154	-34.167	866.6
	1024-bit	**1024-bit**	-0.179	-26.630	848.2
		2048-bit	-0.276	-75.914	783.7
SS 125 °C	**512-bit**	**512-bit**	-0.061	-0.466	942.5
		1024-bit	-0.059	-5.799	944.3
		2048-bit	-0.170	-25.273	854.7
	1024-bit	**1024-bit**	-0.123	-11.748	890.5
		2048-bit	-0.157	-47.016	864.3

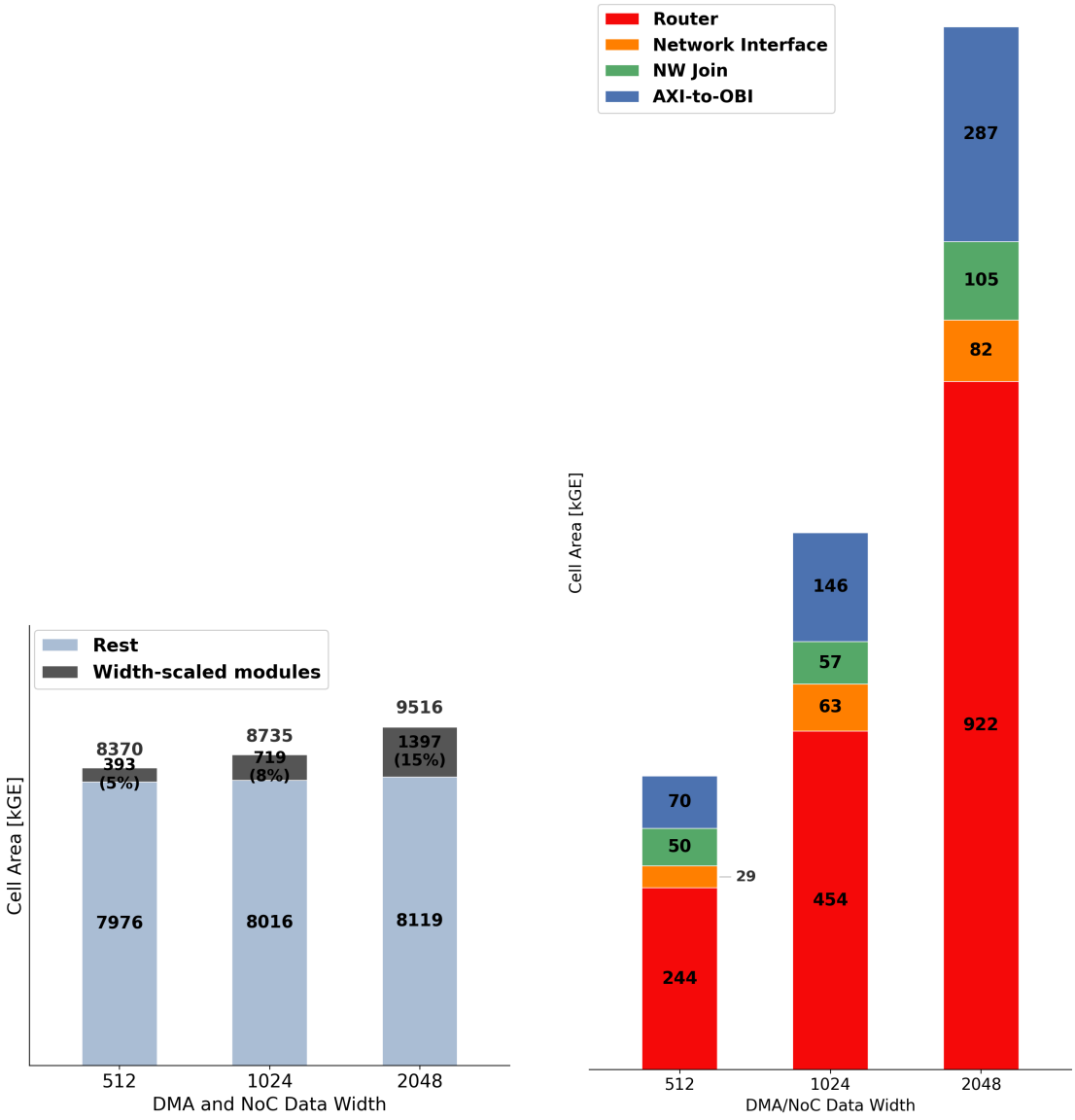
additional routing pressure and placement changes introduced by the wider configurations make timing closure of the existing core logic more difficult.

Operating at a lower data width inside the Snitch cluster therefore helps both timing closure and congestion. The Snitch cluster data width should only be scaled if the performance results require it, as discussed in Chapter 6. In the configurations where a narrower DMA interface is combined with a wider NoC, the design can still benefit from increased network bandwidth while avoiding a large part of the physical-design cost inside the snitch cluster.

5.4. Memory Tile

Because we introduced a new memory controller, we also checked its physical-design impact together with the increased NoC router width. The area results for the memory tile can be seen in Figure 5.4. As before, the left plot shows the total tile area, while the right plot shows the modules that scale with the data width.

The router again scales approximately with the NoC data width. The memory controller, called AXI-to-OBI in the area report, also scales with the data width. This is expected, since it has to handle the full data-width outputs coming from the SRAM. The read-side interleaving controller is included in this AXI-to-OBI block and is therefore not reported separately. However, its area contribution is not expected to be dominated by the data width, since the interleaver only stores AR metadata and does not buffer the read data itself. The main data-width-dependent area increase in the memory controller therefore still comes from the full-width data path. The narrow/wide AXI join, which merges the



Total memory-tile area **Width-scaled module breakdown**

Figure 5.4.: Area results for the evaluated memory-tile configurations.

narrow and wide AXI traffic before the memory controller, also increases with the data width.

Chapter 6

Performance Results

This work evaluates the AXPY and GEMV kernels described in 3.1 for multiple compute-cluster-to-memory-tile ratios. While the evaluated Gwaihir configuration has a ratio of four compute clusters per memory tile, this work also explores ratios of 8:1 and 16:1 to model larger many-core SoCs. This is done by activating 8 and 16 compute clusters during benchmarking while keeping only one memory tile active.

In addition to AXPY and GEMV, a read-only kernel is evaluated. This kernel repeatedly reads chunks of 2048 double-precision elements from memory and is used as a simple upper-bound case for read bandwidth. Since the clusters only synchronize at the beginning of the workload and then share the available NoC bandwidth, it is difficult to define an exact per-iteration runtime for a single AXPY tile, GEMV matrix, or read-only chunk. This is especially true for AXPY, which contains mixed read/write traffic, and for GEMV, where the final matrix computation at the end of the workload slightly disturbs the steady state.

For this reason, the performance numbers are measured in the steady-state regime. The AXPY kernel is run for 40 iterations, the GEMV kernel for 50 iterations, and the read-only kernel for 8 repetitions. The AXPY and read-only kernels operate on two 2048-element double-precision tiles per repetition. The GEMV kernel operates on one 80×80 double-precision matrix and is therefore firmly in the large-matrix regime described in 3.1. Figures 6.1 show the measured speedups for the three compute-cluster-to-memory-tile ratios. The speedups are reported relative to the 512-bit DMA / 512-bit NoC baseline. The evaluated configurations are the ones described in 5.

The results show that simply increasing the NoC data width is not sufficient to reach full speedups in all cases. The read-only, AXPY, and GEMV kernels all benefit from the wider NoC, but especially for the smaller compute-cluster-to-memory-tile ratios, the speedups remain below the ideal scaling expected from the raw increase in link width.

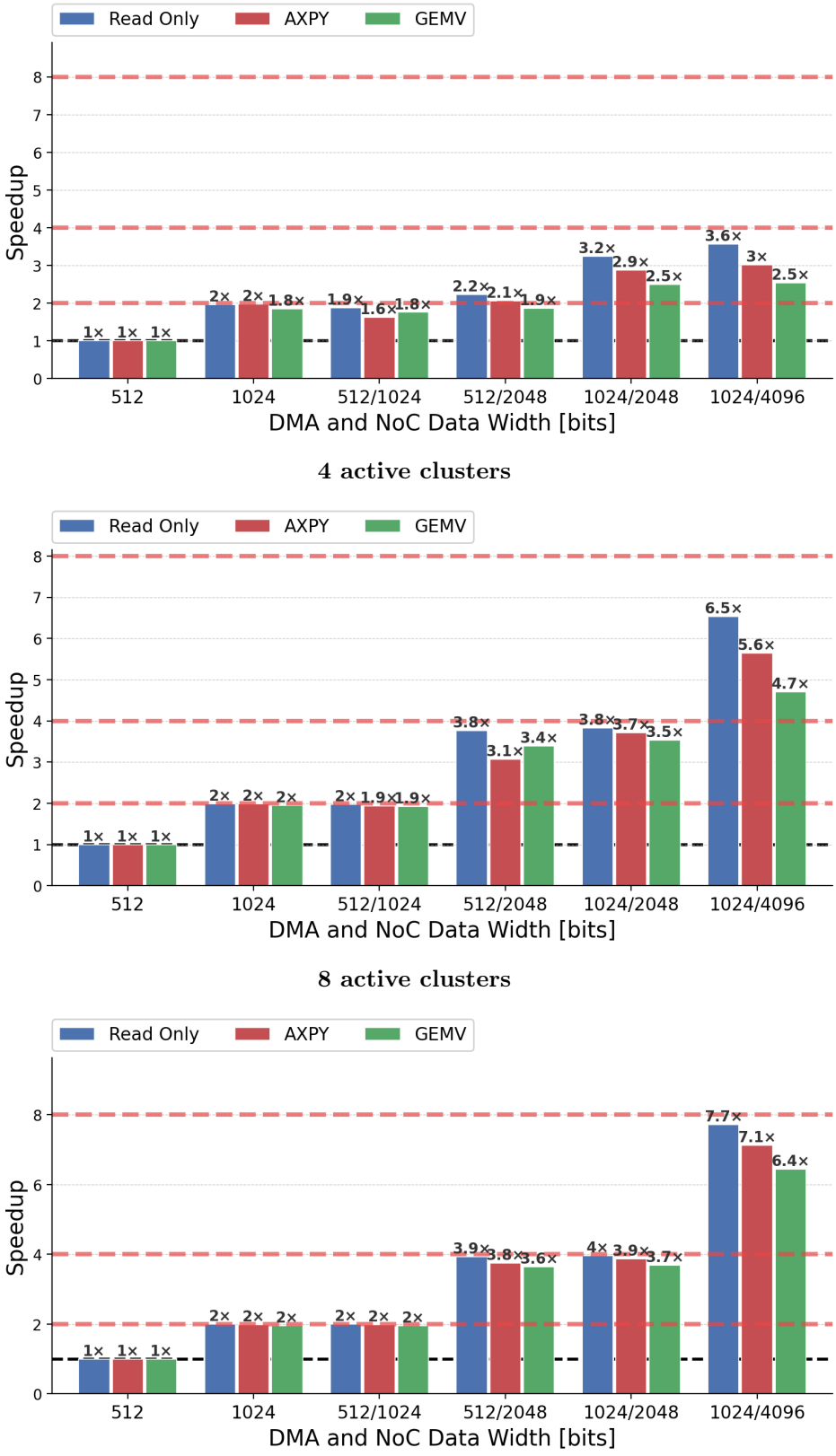


Figure 6.1.: Performance speedups for the evaluated compute-cluster-to-memory-tile ratios. Speedups are normalized to the 512-bit DMA / 512-bit NoC baseline.

The main reason is that traffic can still be serialized at the memory tile and at the narrow cluster-side DMA interface.

This is most visible when the compute-cluster-to-memory-tile ratio is close to the NoC-to-DMA data-width ratio. For example, in the 4:1 case with a 512-bit DMA and a 2048-bit NoC, the memory tile would need four active clusters to fully hide the drain time of each Wide Link Adapter. However, if fewer than all four clusters have an active AR in the interleaver, the interleaver introduced in Section 4.3 cannot fully prevent stalls at the cluster-side Wide Link Adapters.

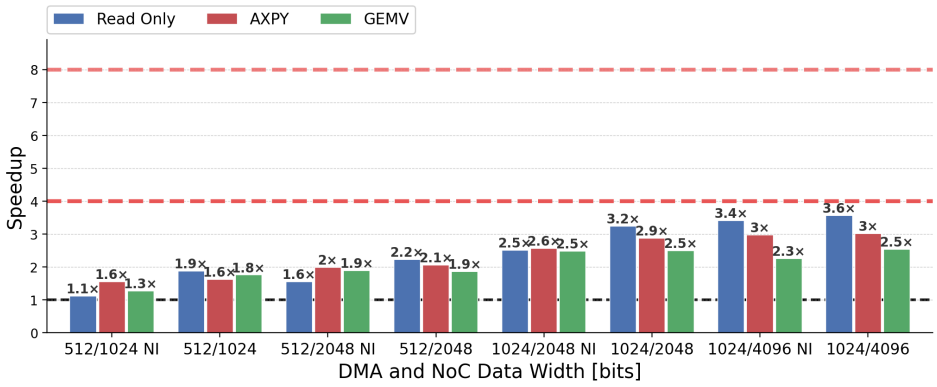
Looking at the waveforms for these benchmarks, the four cluster tiles do not supply ARs fast enough to keep the interleaver fully occupied. After the active ARs have been served, there is a significant dead time before the clusters issue the next read requests. This reduces the effective memory-tile utilization and explains the reduced speedups in the 4:1 configuration, and partly also in the 8:1 configuration. When moving to 16 active clusters, there is enough request pressure to keep the memory tile busy for most of the time, and the measured speedups are much closer to the expected speedup from the NoC scaling factor.

6.1. Impact of Memory-Tile Interleaving

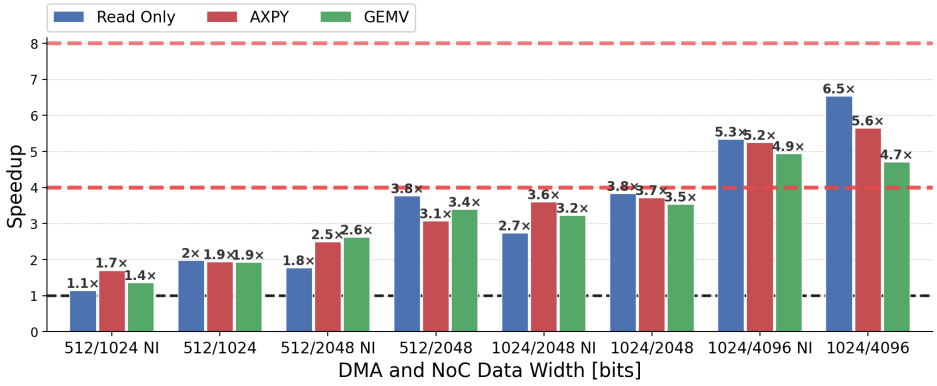
Figures 6.2 compare the speedups with and without the interleaving memory controller for the three evaluated compute-cluster-to-memory-tile ratios. The results show a clear performance benefit across configurations and confirm the assumptions made in 4.3. Instead of serving one long burst until the receiving cluster applies backpressure, the memory tile can interleave active read requests from multiple clusters. This allows another cluster to be served while the previous one drains its local buffer through the narrower DMA interface.

On average, the interleaving controller improves performance by 49.5% for the read-only kernel, 9.2% for AXPY, and 15.8% for GEMV. The read-only kernel benefits most directly, since it only stresses the read path. GEMV also benefits strongly because it is dominated by read traffic in the large-matrix regime. AXPY benefits less, because it contains mixed read and write traffic. Since the current write path is still serialized, only the read part of the traffic can fully benefit from the interleaving controller. However, because AXPY still contains more read than write traffic, a measurable speedup remains.

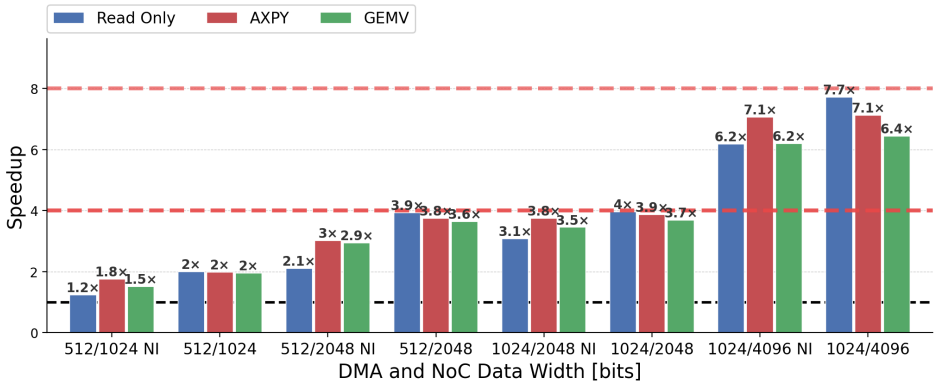
In theory, configurations with a larger ratio between NoC and DMA data width should benefit more from interleaving, because the receiving cluster needs more cycles to drain one wide NoC beat through the narrower DMA interface. This can be seen when comparing the 512-bit DMA / 2048-bit NoC configuration with the 1024-bit DMA / 2048-bit NoC configuration. In the 16-active-cluster experiment, the latter performs significantly better without interleaving. With the improved memory controller, this difference is



4 active clusters



8 active clusters



16 active clusters

Figure 6.2.: Performance speedups for non-interleaving (NI) and interleaving configurations for the evaluated compute-cluster-to-memory-tile ratios.

almost completely removed, indicating that interleaving reduces the serialization bottleneck at the memory tile.

The largest relative improvement is observed for the 512-bit DMA / 1024-bit NoC configuration, while the 1024-bit DMA / 4096-bit NoC configuration shows only a small additional benefit. This is because larger NoC widths reduce the number of beats per 4 kB AXI burst. For example, a 4096-bit NoC transfers only 8 beats per 4 kB burst. In this case, buffering in the routers and the memory tile can hide a larger part of the serialization effect. In contrast, the 512-bit DMA / 1024-bit NoC configuration still has 32 beats per 4 kB burst, so the serialized service time is long enough to noticeably degrade performance without interleaving.

These two effects counteract each other when moving to larger NoC/DMA data-width ratios. A wider NoC increases the drain-time mismatch at the cluster interface, but it also reduces the number of beats per AXI burst and gives the existing buffering more opportunity to hide stalls. The measured results therefore show that memory-tile interleaving is most important when the burst length is still long enough for serialization to dominate, but the NoC is already wide enough to create backpressure at the cluster-side DMA interface.

Chapter 7

Conclusion and Future Work

This work demonstrates that increasing the NoC width is feasible and can bring substantial speedups for memory-bound kernels. Moving to a 1024-bit data width is possible both in the DMA and in the NoC. However, extending the link width further to 2048 bit already requires decoupling the two widths so that the physical design can still be routed. In general, the DMA data width inside the Snitch cluster should be kept as low as possible, since increasing it is the main driver of timing degradation and routing pressure inside the cluster tile.

The performance results show, however, that a large ratio between NoC and DMA data width can degrade performance in smaller compute meshes. This can be seen in the 4:1 compute-cluster-to-memory-tile ratio of Gwaihir, where the speedups drop well below the ideal scaling factor of the NoC. From the evaluated configurations, it seems that the compute-cluster-to-memory-tile ratio should be at least twice as large as the NoC-to-DMA data-width ratio to fully benefit from the wider NoC.

These considerations should be taken into account when deciding by how much the NoC should be scaled. A wider NoC improves the available memory bandwidth, but the benefit depends strongly on the number of clusters sharing one memory tile, the DMA width inside the cluster, and the ability of the memory tile to interleave traffic between multiple outstanding requests.

An important direction for future work is to reduce the need for this extra ratio headroom. In principle, it should be possible to utilize the full NoC bandwidth already when the compute-cluster-to-memory-tile ratio matches the NoC-to-DMA width ratio. For this, the DMA should be able to issue multiple outstanding read requests to the memory tile in parallel. For example, for a larger transfer of around 40 kB, the individual 4 kB AXI bursts could cycle through different AXI IDs. The memory tile could then keep several bursts from the same cluster active at the same time and interleave them with requests from other clusters.

To support this, two main Register Transfer Level (RTL) changes would be required. First, the meta buffer in the memory tile should remap incoming AR requests to a combination of the source ID and the original AXI ID. This would allow the AR slot buffer to distinguish multiple outstanding requests from the same cluster, while still preserving the ordering requirements of AXI for requests with the same ID. Second, the DMA would need to issue consecutive AXI bursts with different IDs, for example by cycling through a small set of available IDs.

With these changes, the pressure on the memory tile could be increased and the phases where the memory tile is idle because no suitable AR is available in the slot buffer would be reduced. This should allow the interleaving controller to better utilize the wide NoC link, especially in configurations where the current design is still limited by the narrow cluster-side DMA interface.

Appendix A

Task Description

Evaluation of On-Chip Link Width in the Picobello SoC (M,S)

From iis-projects

Jump to: navigation, search

Contents

1 Overview
  1.1 Status: In Progress
2 Introduction
3 Project description
  3.1 Detailed task description
  3.2 Stretch goals
  3.3 Character
  3.4 Prerequisites
4 References

Overview

Status: In Progress

- Type: Master or Semester Thesis
- Professor: Prof. Dr. L. Benini
- Supervisors:
  - Lorenzo Leone: lleone@iis.ee.ethz.ch
  - Luca Colagrande: colluca@iis.ee.ethz.ch
- Student: Lars Kröger lkroeger@student.ethz.ch

Introduction

The continued shift toward highly parallel computing systems has made scalable and efficient on-chip communication a first-order design constraint. As core counts increase and heterogeneous accelerators become commonplace, the network-on-chip (NoC) is no longer a secondary infrastructure component but a central determinant of overall system performance, energy efficiency, and scalability. Modern many-core architectures therefore require interconnects that can simultaneously sustain high aggregate bandwidth while providing low-latency support for control and synchronization traffic.

In the IIS group, we developed FlooNoC [1], a versatile AXI-based network-on-chip architecture designed to efficiently support heterogeneous traffic on the same interconnect backbone. To address the differing requirements of bulk data transfers and latency-sensitive control messages, FlooNoC integrates distinct communication channels: a 512-bit wide channel optimized for high-throughput data movement, and a 32-bit narrow channel dedicated to low-latency traffic.

The FlooNoC architecture has recently been integrated and taped out in the Picobello system-on-chip (SoC) [2]. Picobello implements a 7×4 mesh topology interconnecting 16 compute clusters based on the Snitch architecture[3]. As application complexity and data-parallel workload intensity continue to grow, interconnect bandwidth increasingly becomes a critical scalability factor. The objective of this project is to investigate the implications of increasing the FlooNoC wide-channel width from 512 bits to 1024 bits in the Picobello SoC, evaluating the resulting impact in terms of area and routing resource utilization, as well as the achievable bandwidth improvements.

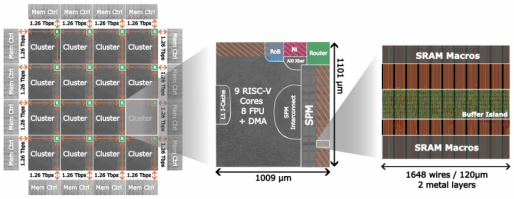


Figure 1: Physical implementation of FlooNoC connecting a mesh of compute tiles in GlobalFoundries' 12 nm technology

Project description

In this thesis, you will increase the FlooNoC wide link width to 1024 bits and adapt the Snitch cluster architecture to support the increased bandwidth. The project includes implementing the required architectural modifications and evaluating their impact on both the compute clusters and the overall Picobello SoC, with particular attention to area, routing resources, timing, and bandwidth scalability.

Detailed task description

To break it down in more detail, you will:

- **Review previous work**:
  - Familiarize yourself with FlooNoC, the Snitch cluster, and the Picobello SoC.
- **Implement support for wider link:**
  - Define and evaluate the most suitable cluster configuration to support a 1024-bit external wide link.
  - Explore PPA overheads and correlate them with your RTL modifications.
- **System integration:**
  - Extend the FlooNoC architecture to support a 1024-bit wide link.
  - Extend the Picobello SoC to integrate your modifications.
  - Analyze and evaluate the implications on area, timing, and routing resources.

Stretch goals

Additional optional stretch goals may include:

- Extend the link width beyond 1024-bit.

Character

- 10% Literature/architecture review
- 20% RTL design and verification
- 60% Physical design exploration
- 10% Bare-metal software development

Prerequisites

- Strong interest in computer architecture
- Experience with digital design in SystemVerilog as taught in VLSI I
- Experience with ASIC implementation flow as taught in VLSI II
- Preferred: Experience in bare-metal or embedded C programming

References

[1] FlooNoC: A 645-Gb/s/link 0.15-pJ/B/hop Open-Source NoC
[2] Picobello SOC
[3] Snitch: A Tiny Pseudo Dual-Issue Processor
[4] FlooNoC repository
[5] Snitch Cluster repository

Appendix B

Full Area Breakdown

DMA	NoC	Data	Block	Total	Δ	Comb.	Seq.	Share
512-bit	**512-bit**	actual	Total	5424	+0.0%	2254	1721	100.0%
			Router	363	+0.0%	130	233	6.7%
			Network Interface	60	+0.0%	13	47	1.1%
			DMA Crossbar	113	+0.0%	28	85	2.1%
			TCDM Interconnect	234	+0.0%	225	9	4.3%
	1024-bit	actual	Total	5806	+7.0%	2420	1936	100.0%
			Router	624	+71.9%	227	397	10.7%
			Network Interface	78	+30.7%	16	61	1.3%
			Wide Link Extender	65	–	31	34	1.1%
			DMA Crossbar	115	+1.5%	30	85	2.0%
			TCDM Interconnect	249	+6.3%	240	9	4.3%
1024-bit	**1024-bit**	actual	Total	5908	+8.9%	2451	2006	100.0%
			Router	610	+68.0%	216	394	10.3%
			Network Interface	78	+30.3%	16	61	1.3%
			DMA Crossbar	201	+77.6%	49	152	3.4%
			TCDM Interconnect	276	+18.0%	267	9	4.7%
	2048-bit	actual	Total	6795	+25.3%	2919	2426	100.0%
			Router	1181	+225.5%	457	724	17.4%
			Network Interface	118	+99.0%	28	91	1.7%
			Wide Link Extender	118	–	60	58	1.7%
			DMA Crossbar	206	+82.3%	54	152	3.0%
			TCDM Interconnect	307	+31.2%	298	9	4.5%
	4096-bit	scaled	Total	8565	+57.9%	3946	3207	100.0%
			Router	2570	+608.0%	1231	1385	30.0%
			Network Interface	209	+251.2%	60	150	2.4%
			Wide Link Extender	211	–	107	104	2.5%
			DMA Crossbar	213	+88.6%	61	153	2.5%
			TCDM Interconnect	330	+41.1%	322	9	3.9%
2048-bit	**2048-bit**	scaled	Total	6952	+28.2%	2914	2588	100.0%
			Router	1175	+223.7%	458	722	16.9%
			Network Interface	120	+102.4%	30	91	1.7%
			DMA Crossbar	389	+243.9%	101	288	5.6%
			TCDM Interconnect	285	+21.9%	276	9	4.1%

Table B.2.: Full area breakdown for the evaluated Snitch cluster-tile configurations. Areas are given in kGE. The percentage increase is relative to the corresponding block in the 512-bit DMA / 512-bit NoC baseline. "actual" denotes route-detail data, "scaled" denotes CTS data scaled with the average route-detail/CTS factor. This was done since we were not able to actually route these configurations. We can assume that these area estimates are rather optimistic, since the modules in these hard-to-route configurations would on average scale more during routing than the full runs with which the scaling factor was calculated.

Bibliography

[1] T. Fischer, M. Rogenmoser, T. Benz, F. K. Gürkaynak, and L. Benini, "Floonoc: A 645-gb/s/link 0.15-pj/b/hop open-source noc with wide physical links and end-to-end axi4 parallel multistream support," *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 33, no. 4, pp. 1094–1107, 2025.

[2] Z. Jia, B. Tillman, M. Maggioni, and D. P. Scarpazza, "Dissecting the graphcore ipu architecture via microbenchmarking," 2019. [Online]. Available: https://arxiv.org/abs/1912.03413

[3] B. Y. Cho, J. Jung, and M. Erez, "Accelerating bandwidth-bound deep learning inference with main-memory accelerators," 2020. [Online]. Available: https://arxiv.org/abs/2012.00158

[4] S. Riedel, M. Cavalcante, R. Andri, and L. Benini, "Mempool: A scalable manycore architecture with a low-latency shared l1 memory," *IEEE Transactions on Computers*, vol. 72, no. 12, p. 3561–3575, Dec. 2023. [Online]. Available: http://dx.doi.org/10.1109/TC.2023.3307796

[5] F. Zaruba, F. Schuiki, and L. Benini, "Manticore: A 4096-core risc-v chiplet architecture for ultra-efficient floating-point computing," 2020. [Online]. Available: https://arxiv.org/abs/2008.06502

[6] F. Zaruba, F. Schuiki, T. Hoefler, and L. Benini, "Snitch: A tiny pseudo dual-issue processor for area and energy efficient execution of floating-point intensive workloads," *IEEE Transactions on Computers*, vol. 70, no. 11, pp. 1845–1860, 2021.

[7] C. Duan, J. Yang, R. Yang, Y. Wang, Y. Wang, L. Long, Y. Qi, X. He, A. Zhou, and W. Zhao, "Hpim: Heterogeneous processing-in-memory-based accelerator for large language models inference," 2025.

[8] Arm Limited. (2024) AXI Protocol Overview. [Online]. Available: https://developer.arm.com/documentation/102202/0300/AXI-protocol-overview

[9] A. Kurth, W. Rönninger, T. Benz, M. Cavalcante, F. Schuiki, F. Zaruba, and L. Benini, "An open-source platform for high-performance non-coherent on-chip communication," *IEEE Transactions on Computers*, vol. 71, no. 8, pp. 1794–1809, 2022.

[10] Integrated Systems Laboratory, ETH Zurich, "Picobello," http://asic.ethz.ch/2025/Picobello.html, 2025, accessed: 2026-06-18.

[11] L. Benini and G. Micheli, "Networks on chips: A new soc paradigm," *Computer*, vol. 35, pp. 70–78, 02 2002.

[12] T. Benz, M. Rogenmoser, P. Scheffler, S. Riedel, A. Ottaviano, A. Kurth, T. Hoefler, and L. Benini, "A high-performance, energy-efficient modular dma engine architecture," *IEEE Transactions on Computers*, vol. 73, no. 1, pp. 263–277, 2023.

[13] Z. Zhang, D. Parikh, Y. Zhang, and V. Prasanna, "Benchmarking the performance of large language models on the cerebras wafer scale engine," 2024. [Online]. Available: https://arxiv.org/abs/2409.00287